Visual Inertial SLAM

Jasper Ha

March 2025

1 Introduction

SLAM is a very important topic in robotics. LiDar is expensive, while stereo cameras are more affordable. As a result, it is important to be able to perform SLAM with stereo cameras without LiDar to create a map and localize in an environment. The approach will use an IMU and stereo combine the data through the use of Extended Kalman Filter to perform SLAM. The first part will implement dead reckoning of the trajectory using EKF prediction and IMU data. The second part will implement landmark position estimation using EKF update assuming the trajectory from part 1 is the true trajectory. The last part will combine both parts to perform SLAM through the implementation of EKF trajectory update with stereo-camera data.

2 Problem Formulation

We can formulate a motion model for the robot with the following equation below, where $T_t \in SE(3)$ represents the pose of the robot IMU with respect to the world frame at time t. The input is $\zeta(t) = \begin{bmatrix} v(t) \\ \omega(t) \end{bmatrix}$ and $\tau_k = t_{k+1} - t_k$. Where $v(t) \in \mathbb{R}^3$ is the linear velocity at time t and $\omega(t) \in \mathbb{R}^3$ is the angular velocity at time t.

$$T_{k+1} = T_k exp(\tau_k \hat{\zeta}_k)$$

We can also formulate the observation model for the robot with the following equation below, where i and j are coupled by $\Delta_t(j) = i$ which associates landmark positions $\underline{m}_j$ to stereo-camera sensor readings $z_{t,i}$. Similarly, $T_t \in SE(3)$ represents the pose of the robot IMU with respect to the world frame at time t. Ks are the stereo camera intrinsics and ${}_OT_I$ is the transformation from the IMU frame to optical frame. $\pi(q) = \begin{bmatrix} q_1/q_3 \\ q_2/q_3 \\ 1 \end{bmatrix}$ is the projection function which normalizes the depth where $q \in \mathbb{R}^4$. $z_{t,i} \in \mathbb{R}^4$

$$z_{t,i} = K\pi({}_OT_I T_t^{-1} \underline{m}_j)$$

Problem 1: IMU Localization with EKF prediction The problem is given the input $\zeta(t)$ and T_t formulate the EKF prediction for T_{t+1} which means create a function $f(\zeta(t), T_t)$ that estimates the next pose T_{t+1}. Then use the function to iteratively estimate the trajectory of the robot over the time span.

Problem 2: Landmark mapping via EKF The problem is given the true trajectory T_t, formulate the EKF update for the landmark positions $m \in \mathbb{R}^{3M}$ to iteratively update the estimated landmark positions from the sensor measurements $z_t \in \mathbb{R}^{4N}$. Where M is the total number of landmarks and N is the number of visible landmarks at time t

Problem 3: Visual-Inertial SLAM The problem is to fuse the IMU Localization EKF prediction from problem 1 and Landmark mapping via EKF from problem 2 to perform SLAM. The missing component is IMU localization with EKF update with sensor data. Thus, the main subproblem is to formulate the localization update and then fuse with the landmark mapping in problem 2.

3 Technical Approach

Problem 1: IMU Localization with EKF prediction In this problem we are to implement EKF prediction for the IMU localization. For the IMU pose motion model, the pose EKF prediction equation is as follows below.

$T_t|z_{0:t}, u_{0:t-1} \sim N(\mu_{t|t}, \Sigma_{t|t})$ where $\mu_{t|t} \in SE(3)$ and $\Sigma_{t|t} \in \mathbb{R}^{6\times 6}$ We assume that the motion model has noise $w_t \sim N(0, W)$ From this, we define the kinematics

$$\mu_{t+1|t} = \mu_{t|t} exp(\tau_t \hat{u}_t)$$
$$\delta\mu_{t+1|t} = exp(-\tau_t \hat{u}_t)\delta\mu_{t+1|t} + \sqrt{\tau_t} w_t$$

This is because we need to ensure $\mu \in SE(3)$.

EKF IMU Predict Equations

$$\mu_{t+1|t} = \mu_{t|t} exp(\tau_t \hat{u}_t)$$

$$\Sigma_{t+1|t} = exp(-\tau_t \hat{u}_t)\Sigma_{t|t} exp(-\tau_t \hat{u}_t)^T + \tau_t W$$

We only perform a prediction without any sensor data, so there is no update step. This means that $\mu_{t+1|t+1} = \mu_{t+1|t}$ and $\Sigma_{t+1|t+1} = \Sigma_{t+1|t}$. When we estimate our trajectory, we take $\mu_{t|t} = T_t$

Problem 2: Landmark In this approach we first need to be able to convert stereo pixel coordinates to position. This is done by the algorithm below. Given $z_{t,i} = [u_L, v_L, u_R, v_R]^T \in \mathbb{R}^4$ (pixel coordinates), $Ks = \begin{bmatrix} f_{su} & f_{s\theta} & c_u \\ 0 & f_{sv} & c_v \\ 0 & 0 & 1 \end{bmatrix}$ the camera intrinsic matrix and ${}_IT_W$, the pose of the robot at time t, we can find the position of the point.

We first find $d = u_L - v_L$ which is the disparity. From d, we can find the depth l with the equation $-f_v/d$ With the depth found, we can find m_O, the position of the landmark in optical frame

$\underline{m}_O = \begin{bmatrix} l(u_L - c_u)/f_{su} \\ l(v_L - c_v)/f_{sv} \\ l \\ 1 \end{bmatrix}$ Homogeneous coordinates in optical frame $\underline{m}_W = {}_IT_W{}^{-1}{}_OT_I{}^{-1}m_O$ Homogeneous coordinates in world frame

In this problem we need to first filter the sensor data to discard data with high error. This can be from error in landmark pixel coordinates in either left or right camera causing the depth to be off. This can cause estimated landmark positions to be behind the robot or far away.

We can solve the issue by constraining valid points on the depth and distance from the robot $l > 0$ for positive depth meaning in front of the robot. Also, $U > ||m_W - p|| > L$ where $p \in \mathbb{R}^3$ is the position of the robot in the world frame, m_W is the landmark position in world frame. U is the upperbound on the distance and L is the lowerbound. We need a lowerbound and upperbound because at extreme close or far distances, there can be high error from sensor measurements, especially with pixel matching.

After filtering for good sensor data, we have $z_t \in \mathbb{R}^{4N_t}$ where N_t is the number of good sensor readings at the time t.

The EKF update takes in $\mu_t \in \mathbb{R}^{3M}$ the landmark position estimates at time t where M is the number of landmarks, $\Sigma_t \in \mathbb{R}^{3M \times 3M}$ is the covariance matrix for the landmark position estimates at time t, $z_{t+1} \in \mathbb{R}^{4N_{t+1}}$ where N_{t+1} is the number of good sensor readings at the time. It is also assumed that we know the camera intrinsics Ks, ${}_OT_I$ the camera extrinsics, T_{t+1} next pose.

Since we assume that the IMU trajectory is true, we do not need to perform prediction for the observation.

We can compute the the observation model jacobian H_{t+1}

$$H_{t+1,i,j} = K_s \frac{d\pi}{dq}({}_OT_IT_{t+1}{}^{-1}\underline{\mu}_{t,j}){}_OT_IT_{t+1}{}^{-1}P^T, \Delta_t(j) = i$$

$$H_{t+1,i,j} = 0, \Delta_t(j) \neq i$$

We can then compute the Kalman gain and updated mean and covariance where $\tilde{z}_{t+1} = K\pi({}_OT_IT_t^{-1}\underline{m}_{t+1})$ are the corresponding pixel coordinates from passing m_{t+1}. $V \in \mathbb{R}^4$ is the observation model noise covariance matrix.

$$K_{t+1} = \Sigma_t H_{t+1}^T (H_{t+1}\Sigma_t H_{t+1}^T + I \otimes V)^{-1}$$
$$\mu_{t+1} = \mu_t + K_{t+1}(z_{t+1} - \tilde{z}_{t+1})$$
$$\Sigma_{t+1} = (I - K_{t+1}H_{t+1})\Sigma_t$$

We initialize μ_0 as the first good sensor measurements for each corresponding landmark. With μ_0 we can iteratively update using the equations above where z_{t+1} are the filtered observations.

Problem 3: Visual-inertial SLAM

In this problem we need to form $H_{L,t+1}$ the jacobian for the IMU localization update at time $t+1$

$$H_{L,t+1} = \begin{bmatrix} H_{L,t+1,1} \\ \vdots \\ H_{L,t+1,N_{t+1}} \end{bmatrix}$$

$$H_{L,t+1,i} = -K_s \frac{d\pi}{dq}({}_O T_I \mu_{t+1|t}{}^{-1} \underline{m}_{t,j})_O T_I (\mu_{t+1|t}{}^{-1} \underline{m}_{t,j})^{\odot}$$

Now that we have the jacobian, we can combine the EKF pose update and EKF map update into a single EKF update.

The Visual Inertial SLAM algorithm for EKF is as follows

Let μ_M represent the map estimate mean, μ_P represents the pose mean, $\Sigma \in \mathbb{R}^{6+3M \times 6+3M}$ represents covariance matrix of the entire process, $V \in \mathbb{R}^4$ is the observation noise covariance matrix, H_P be the pose jacobian and H_M be the map jacobian.

1. Estimate $\mu_{P,t+1|t}$ using Problem 1
2. Calculate $H_{P,t+1}$ using the estimated next pose $\mu_{P,t+1|t}$
3. Calculate $H_{M,t+1}$ using the estimated next pose $\mu_{P,t+1|t}$ using Problem 2
4. Form $H_{t+1} = \begin{bmatrix} H_{P,t+1} & H_{M,t+1} \end{bmatrix} \in \mathbb{R}^{4N_{t+1} \times 6+3M}$
5. Calculate the Kalman gain

$$K_{t+1} = \Sigma_{t+1|t} H_{t+1}^T (H_{t+1} \Sigma_{t+1|t} H_{t+1}^T + I \otimes V)^{-1} = \begin{bmatrix} K_{P,t+1} \in \mathbb{R}^{6 \times 4N_{t+1}} \\ K_{M,t+1} \in \mathbb{R}^{3M \times 4N_{t+1}} \end{bmatrix}$$

6. Update Pose mean

$$\mu_{P,t+1|t+1} = \mu_{P,t+1|t} exp((K_{P,t+1}(z_{t+1} - \tilde{z}_{t+1}))^{\wedge})$$

7. Update Landmarks mean

$$\mu_{M,t+1|t+1} = \mu_{M,t+1|t} + K_{M,t+1}(z_{t+1} - \tilde{z}_{t+1})$$

8. Update Covariance Matrix

$$\Sigma_{t+1|t+1} = (I - K_{t+1} H_{t+1}) \Sigma_{t+1|t}$$

4 Results

Dataset 00:

Problem 1:

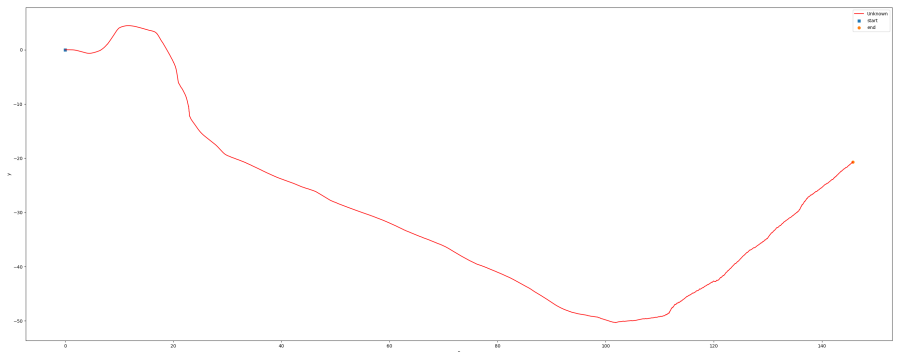

Figure 1: Dataset 00 IMU EKF Prediction Trajectory

Problem 2:

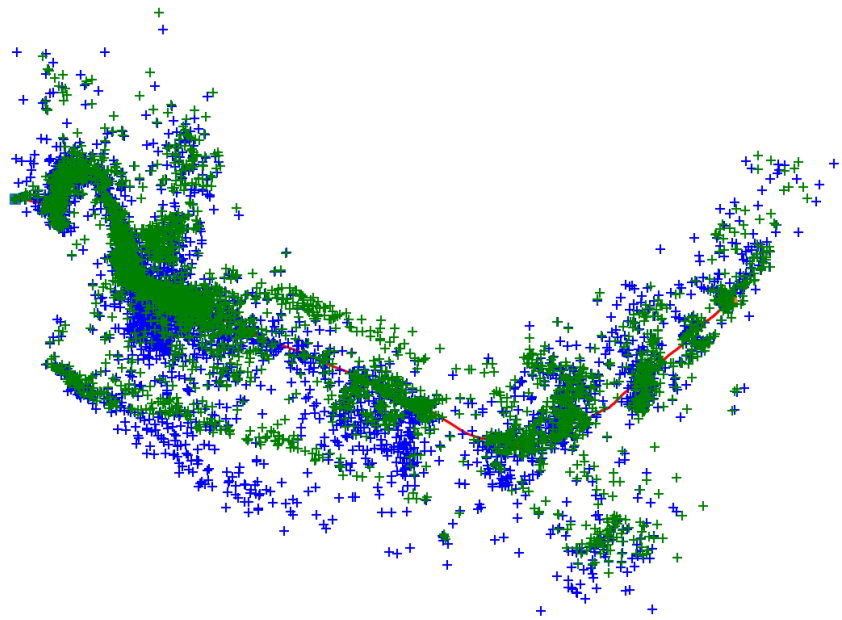

Figure 2: Initialization vs After Updates (Green Landmarks Initial vs Blue Landmarks After Update)

Problem 3:

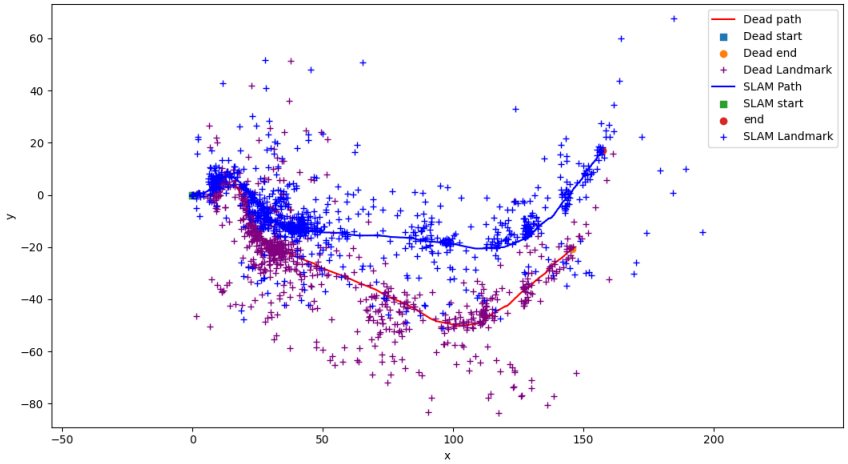

Figure 3: SLAM with 1000 evenly sampled landmarks

Dataset 01:
Problem 1:

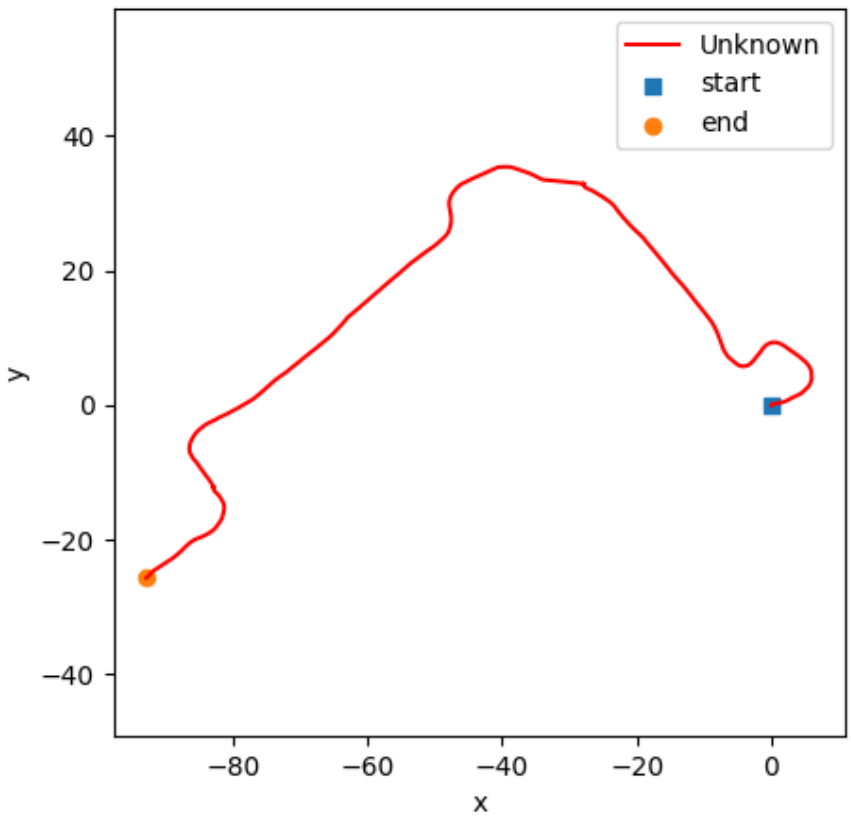

Figure 4: Problem 1 plot for Dataset 01

Problem 2:
1000 Landmarks evenly chosen

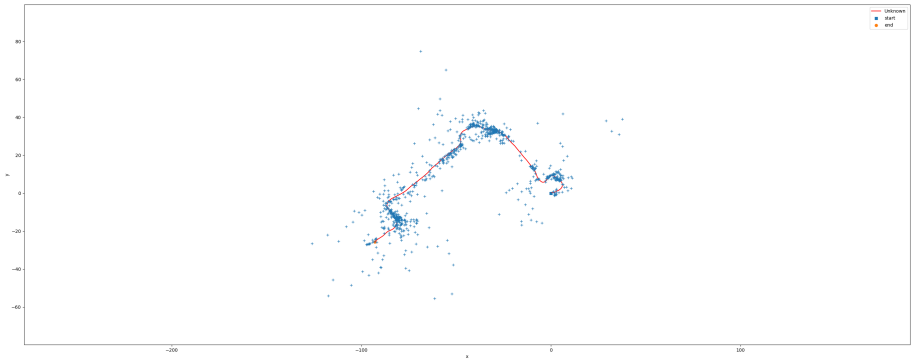

Figure 5: Landmark Initialization

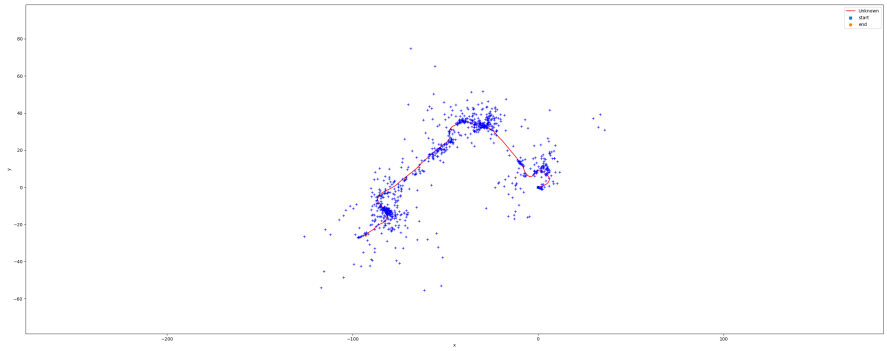

Figure 6: Landmark After Updates

All Landmarks

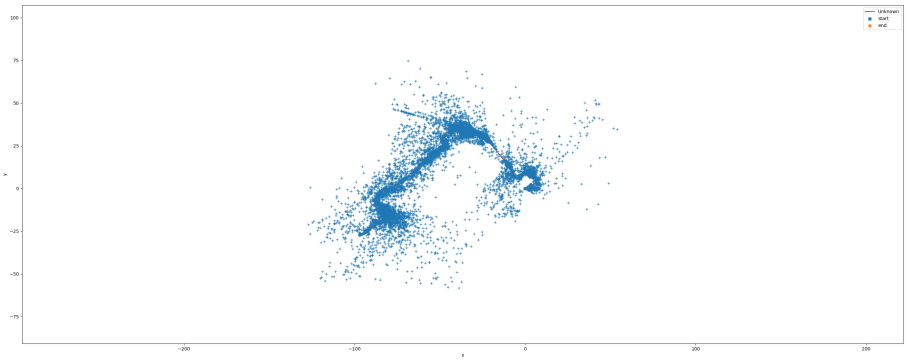

Figure 7: Landmark Initialization

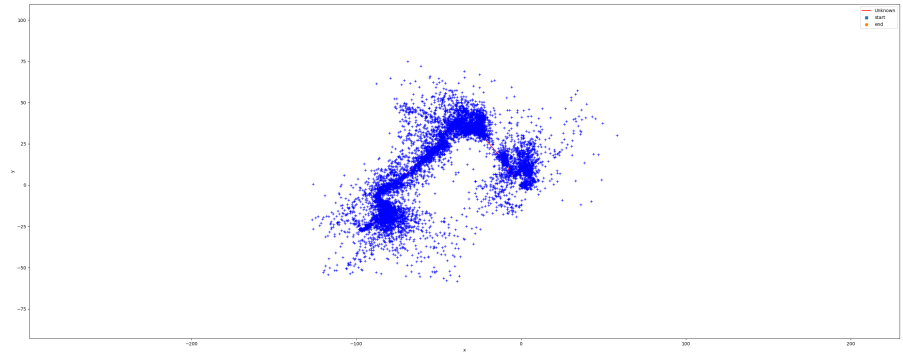

Figure 8: Landmark After Update

Problem 3:

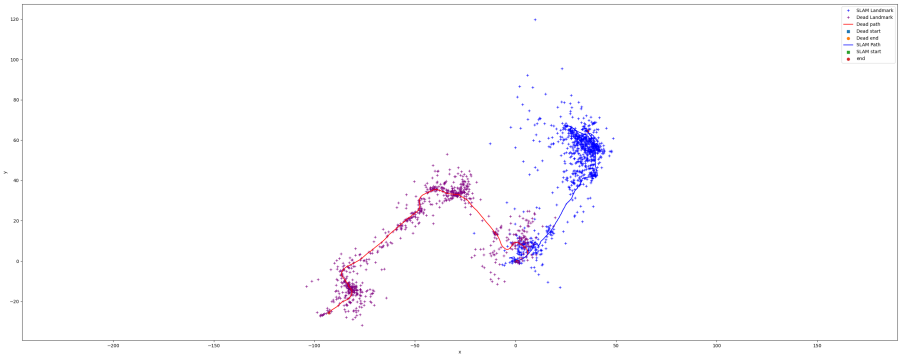

Figure 9: Even Sampling of 1000 landmarks

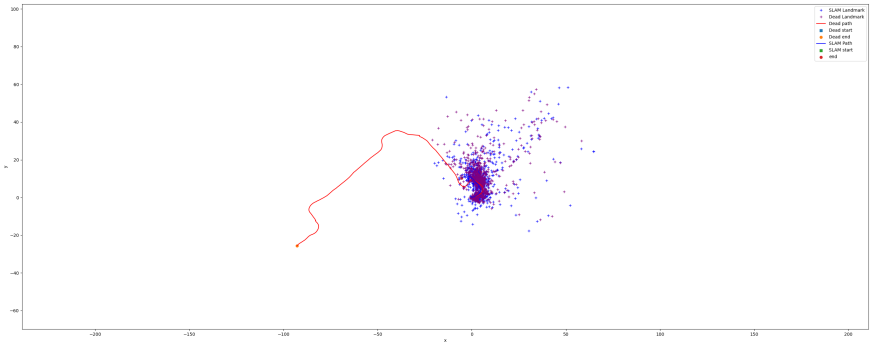

Figure 10: All Landmarks for first 300 steps

Analysis:

I notice that the SLAM algorithm relies heavily on having the correct bad sensor data rejection and needs to be tuned to the environment. For example, for data set 00, the environment is more spacious so the constraints of U needed to be increased to about 30 to ensure the landmarks on the buildings are captured, while reducing the amount of noise from the erroneous sensor data. It is also heavily dependent on the density of points on staying on the correct trajectory. For data set 01, when I reduced the amount of landmarks evenly to 1000, my trajectory turned in a different direction. This may be from the landmarks staying in frame being next to each other, so when sampled evenly, we lose a lot of landmark density, causing error. I also notice that when the V matrix entry increases, there is less spread of the landmarks as opposed to small V values. This may be from a larger dependence on the sensor measurements than the estimated landmark position. Since the stereocamera sensors are very noisy, small V entries the estimated landmark positions to spread out very far. The filter also performs better when the W matrix for the mean covariance entries are small, so there is more trust on the initialized points for problem 2.

Improvements can be to identify large densities of landmarks close to each other. This can help reduce effect of error from outliers on the trajectory.